THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY (GUANGZHOU)

THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

AIAA 2290: Ethics, Privacy and Security in AI

Defending AI Systems

Xuming HU

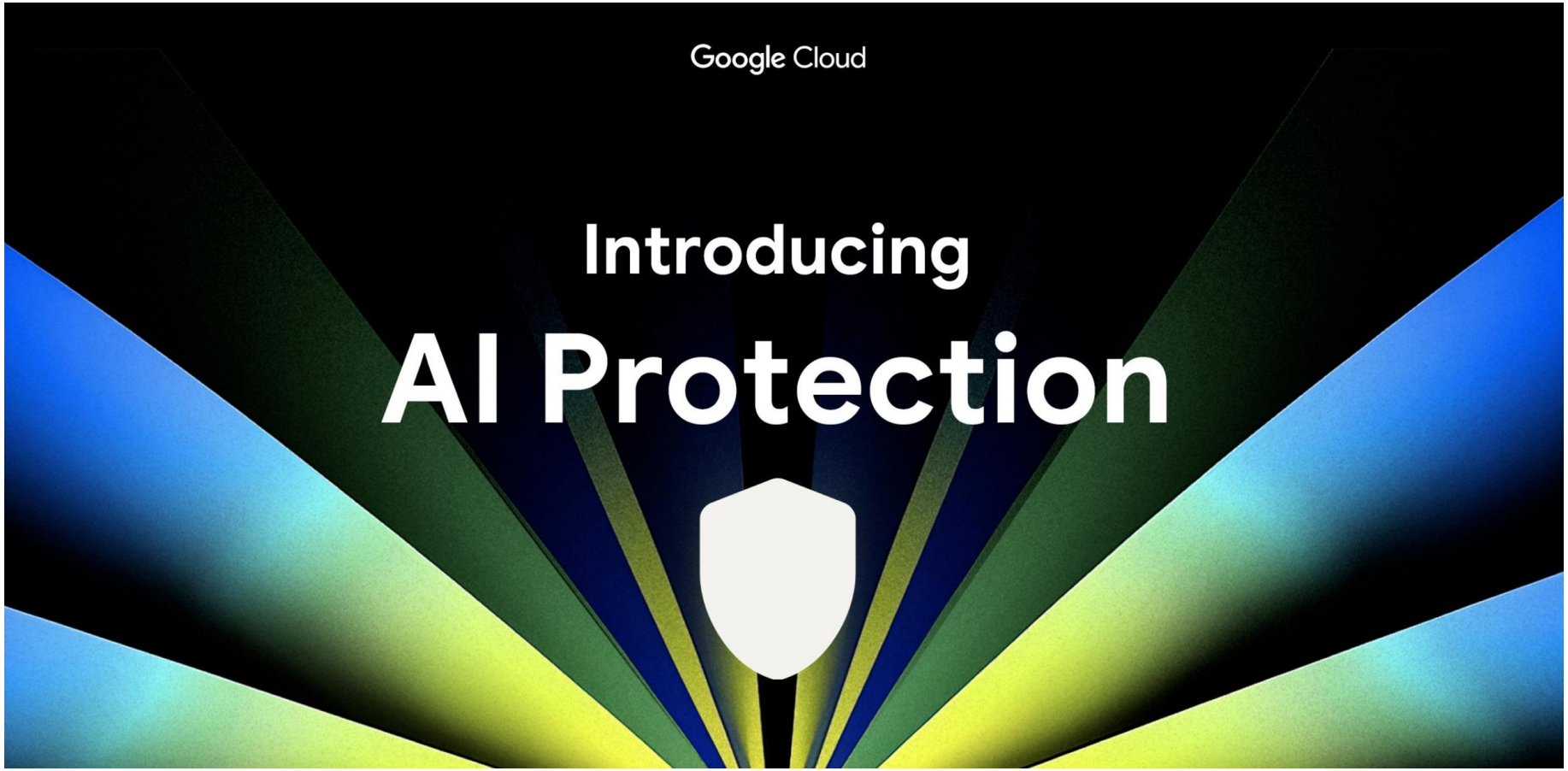
xuminghu@hkust-gz.edu.cn

The Hong Kong University of Science and Technology (Guangzhou)

2025 Fall

Outline

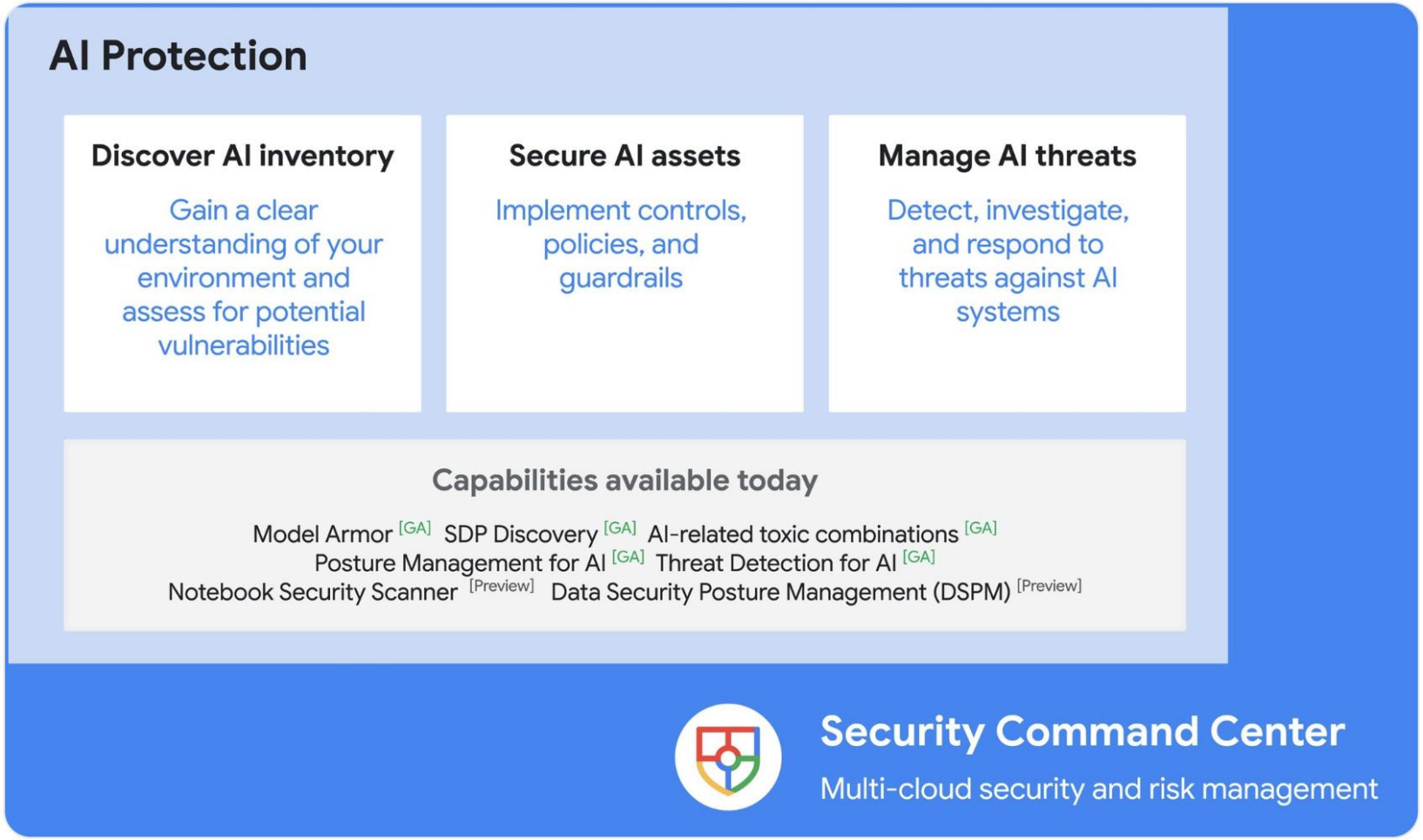
An Example of Google Cloud's AI Protection System.



An Example of Google Cloud's AI Protection System.

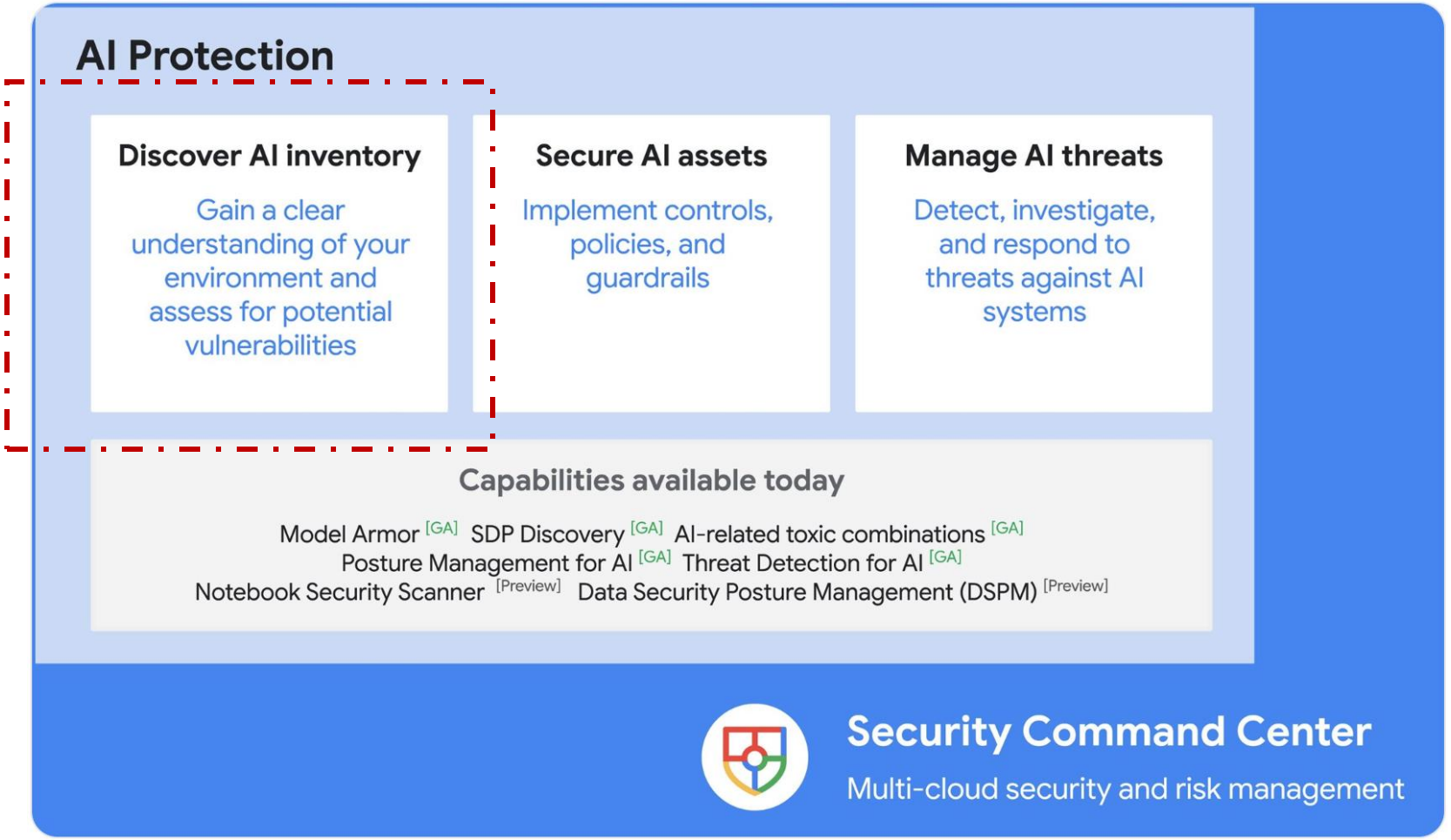
An Example of Google Cloud's AI Protection System.

AI Protection helps organizations discover AI inventory, secure AI assets, and manage AI threats, and is integrated with Security Command Center.



An Example of Google Cloud's AI Protection System.

AI Protection helps organizations discover AI inventory, secure AI assets, and manage AI threats, and is integrated with Security Command Center.



An Example of Google Cloud's AI Protection System.

Discovering AI inventory

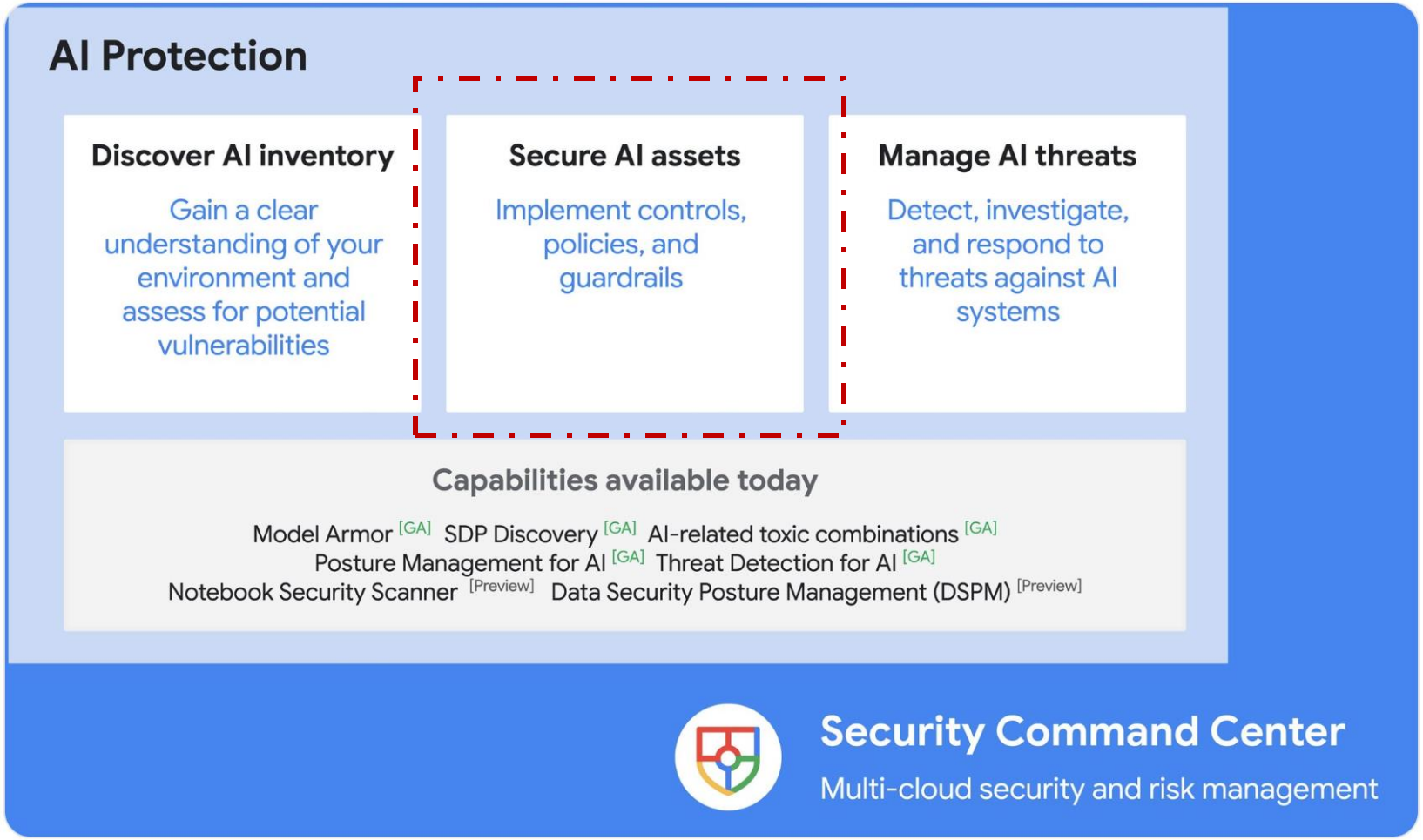
Effective AI risk management begins with a comprehensive understanding of where and how AI is used within your environment. Our capabilities help you automatically discover and catalog AI assets, including the use of ***models, applications, and data — and their relationships***.

Once you know where sensitive data exists, ***AI Protection can use Security Command Center's virtual red teaming to identify AI-related toxic combinations and potential paths that threat actors could take to compromise this critical data***, and recommend steps to remediate vulnerabilities and make posture adjustments.

$300 in free credit to try Google Cloud security products

An Example of Google Cloud's AI Protection System.

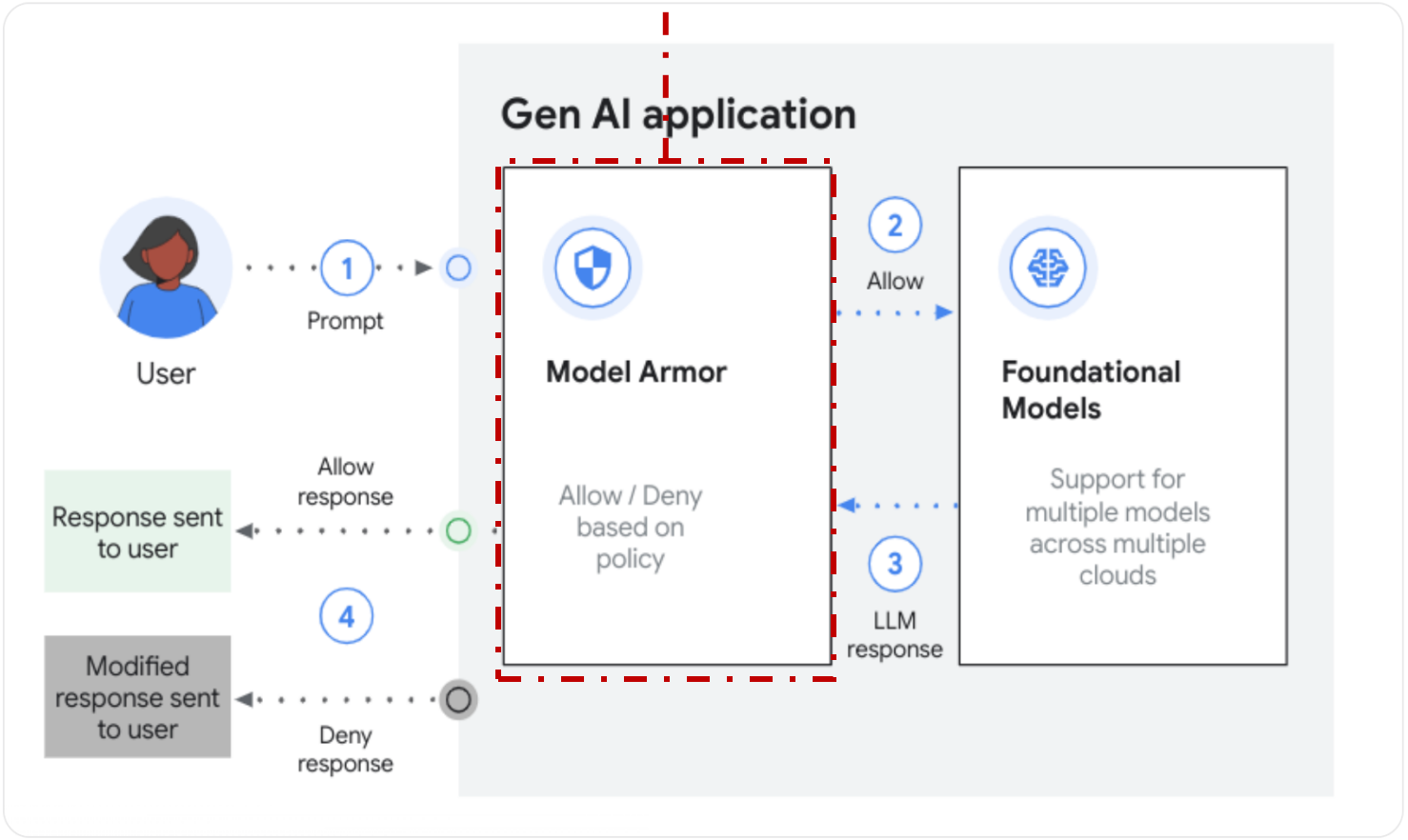
AI Protection helps organizations discover AI inventory, secure AI assets, and manage AI threats, and is integrated with Security Command Center.



An Example of Google Cloud's AI Protection System.

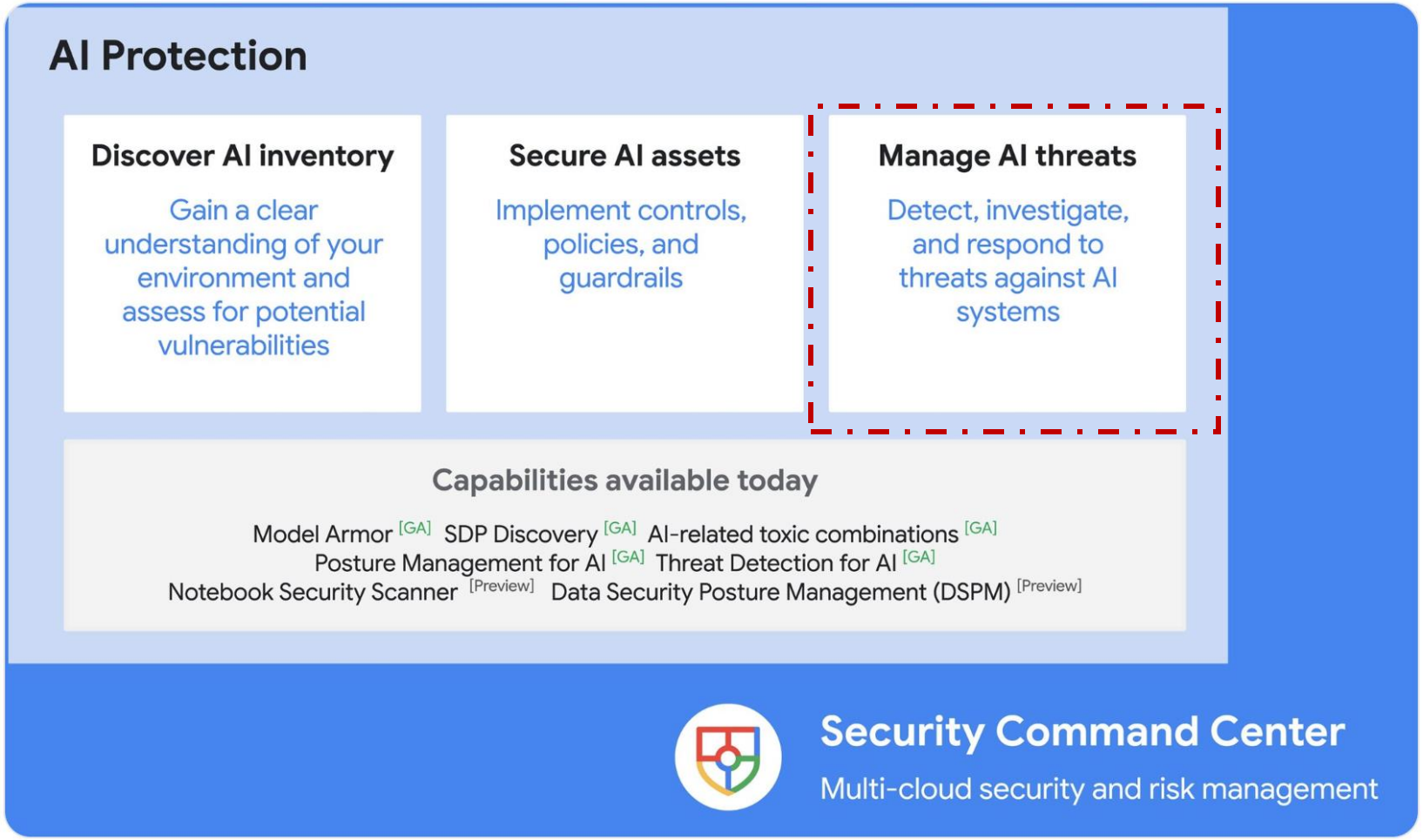
Securing AI assets

Model Armor, a core capability of AI Protection, is now generally available. It guards against prompt injection, jailbreak, data loss, malicious URLs, and offensive content. Model Armor can support a broad range of models across multiple clouds, so customers get consistent protection for the models and platforms they want to use — even if that changes in the future.



An Example of Google Cloud's AI Protection System.

AI Protection helps organizations discover AI inventory, secure AI assets, and manage AI threats, and is integrated with Security Command Center.



An Example of Google Cloud's AI Protection System.

Managing AI threats

AI Protection operationalizes security intelligence and research from Google and Mandiant to help defend your AI systems. Detectors in ***Security Command Center*** can be used to uncover initial access attempts, privilege escalation, and persistence attempts for AI workloads. New detectors to AI Protection based on the latest frontline intelligence to help identify and manage runtime threats such as ***foundational model hijacking are coming soon..***

"*As AI-driven solutions become increasingly commonplace, securing AI systems is paramount and surpasses basic data protection. AI security — by its nature — necessitates a holistic strategy that includes model integrity, data provenance, compliance, and robust governance*,"

—— Dr. Grace Trinidad, research director, IDC.

Adversarial Training

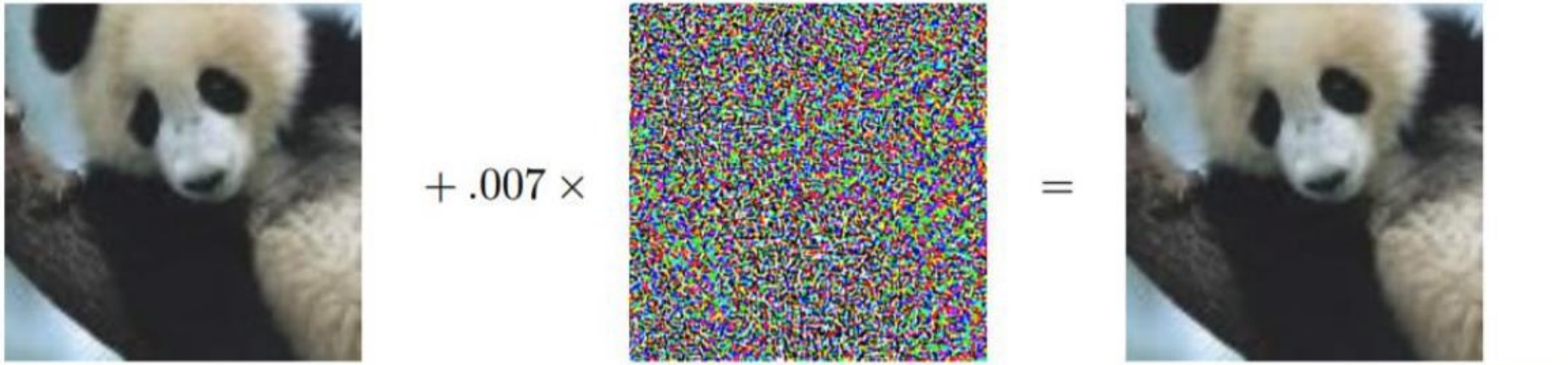
Deep Neural Network

Stop sign

Deep Neural Network

Speed Limit 45

Adversarial Training



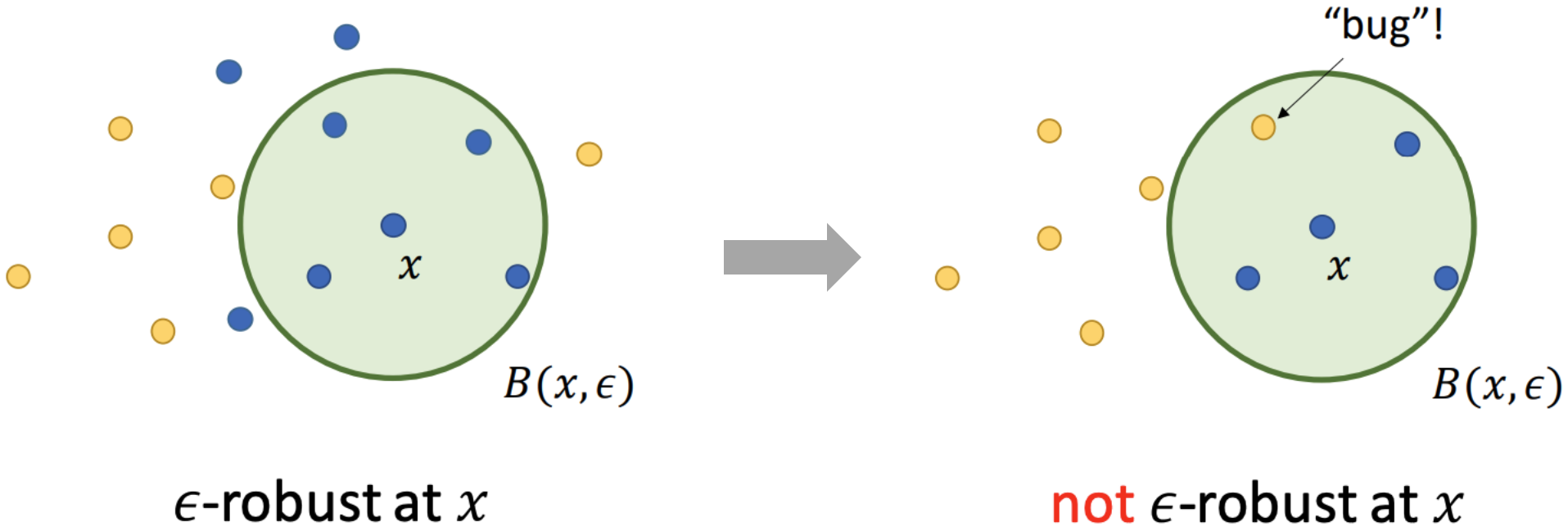
Adversarial Training

Robustness: similar images ⇒ same label

Adversarial Training

$$\textbf{Robustness: } ||x - x'|| \infty \leq \epsilon \Rightarrow \textbf{same label}$$

Adversarial Training



ϵ-robust at x

not ϵ-robust at x

Adversarial Training

- **Given a model f parameterized by θ**

Adversarial Training

- Given a model f parameterized by θ
- Loss(x, y; θ) denotes the error of f_θ on input x with respect to desired output y

Adversarial Training

- Given a model f parameterized by θ
- Loss(x, y; θ) denotes the error of f_θ on input x with respect to desired output y
- Given training set S of labeled input/output examples (x, y)

Adversarial Training

- Given a model f parameterized by θ
- Loss(x, y; θ) denotes the error of f_θ on input x with respect to desired output y
- Given training set S of labeled input/output examples (x, y)
- Goal: Account for adversarial examples during learning (update of parameters θ)

Adversarial Training

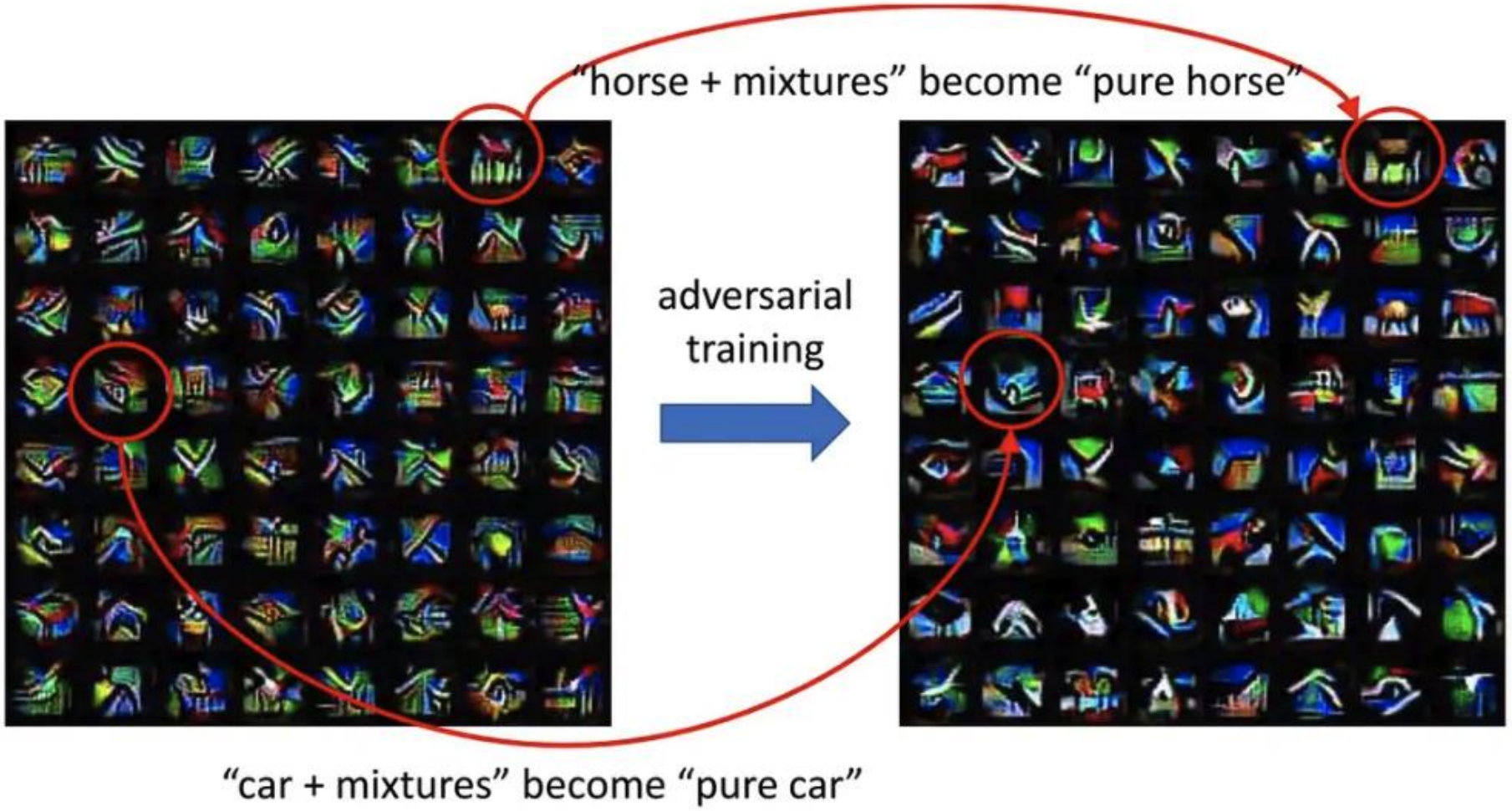
- Given a model f parameterized by θ
- Loss(x, y; θ) denotes the error of f_θ on input x with respect to desired output y
- Given training set S of labeled input/output examples (x, y)
- Goal: Account for adversarial examples during learning (update of parameters θ)
- Adversarial training as optimization:

$$\min_{\theta} \sum_{x,y \in S} \max_{\delta \in \Delta} \text{Loss}\left(x + \delta, y; \theta\right)$$

Adversarial Training

How Adversarial Training Work?

Key Point: "*Purification*" of learned features.

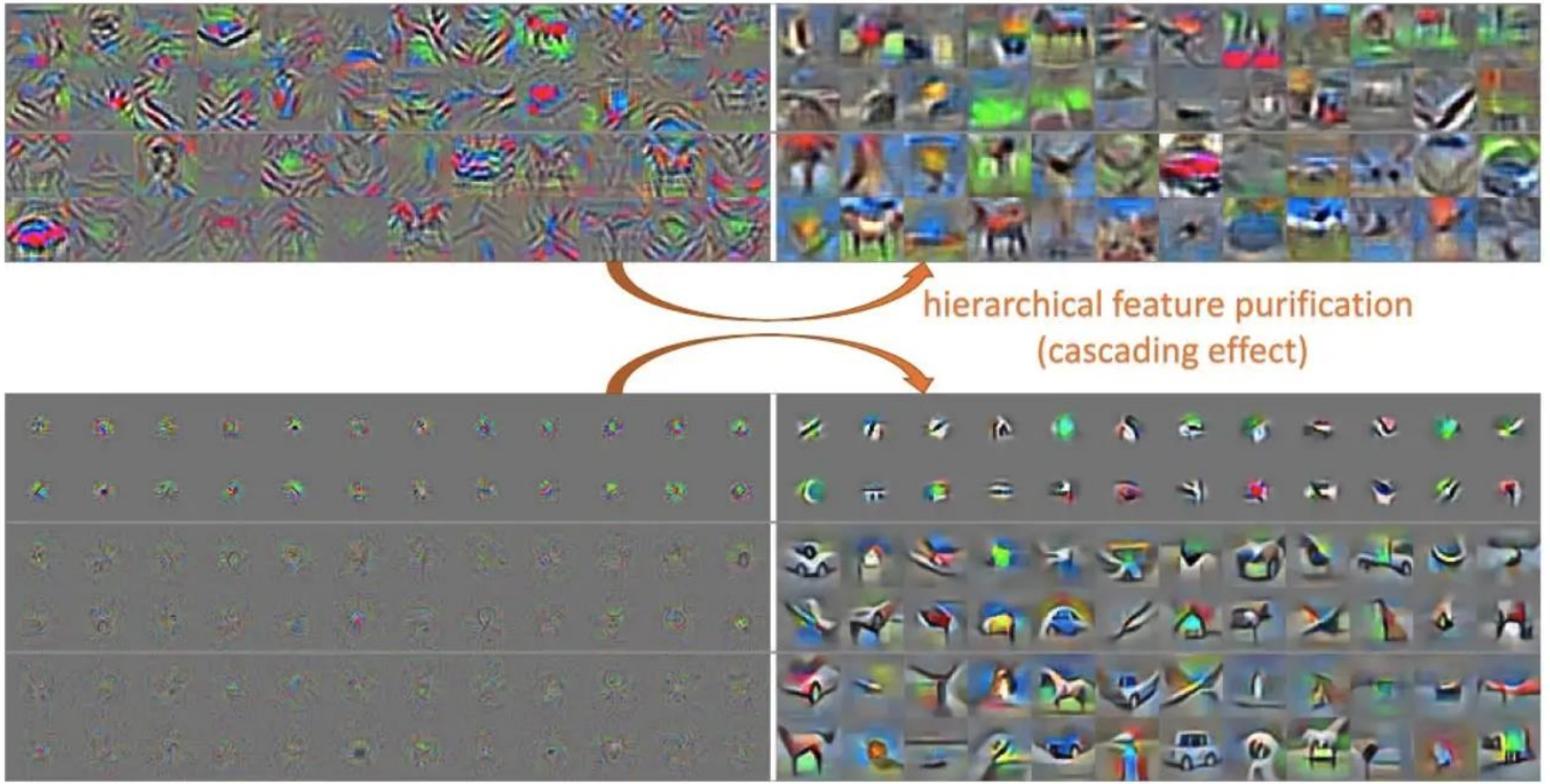


Adversarial Training

How Adversarial Training Work?

Key Point: "*Purification*" of learned features.

(a) AlexNet feature visualization, layers 2 + 4, clean (left) vs. robust (right)



(b) ResNet-34 feature visualization, layers 15 + 25 + 29, clean (left) vs. robust (right)

Model Encryption

Model Encryption

Therefore, the most important way to ensure the security of the model is to increase the robustness of the model to face attacks from adversarial samples.

Model Encryption

Encryption for LLM

Despite the appeal of LLMs, privacy concerns persist surrounding user queries that are processed by these models. ***On the one hand, leveraging the power of LLMs is desirable, but on the other hand, there is a risk of leaking sensitive information to the LLM service provide***r. In some areas, such as ***healthcare, finance, or law***, this privacy risk is a showstopper.

One possible solution to this problem is on-premise deployment, where the LLM owner would deploy their model on the client's machine. ***This is however not an optimal solution, as building an LLM may cost millions of dollars (4.6M$ for GPT3) and on-premise deployment runs the risk of leaking the model intellectual property (IP).***

Zama believes you can get the best of both worlds: our ambition is to protect both the privacy of the user and the IP of the model. In this blog, you'll see how to leverage the Hugging Face transformers library and have parts of these models run on encrypted data.

Model Encryption

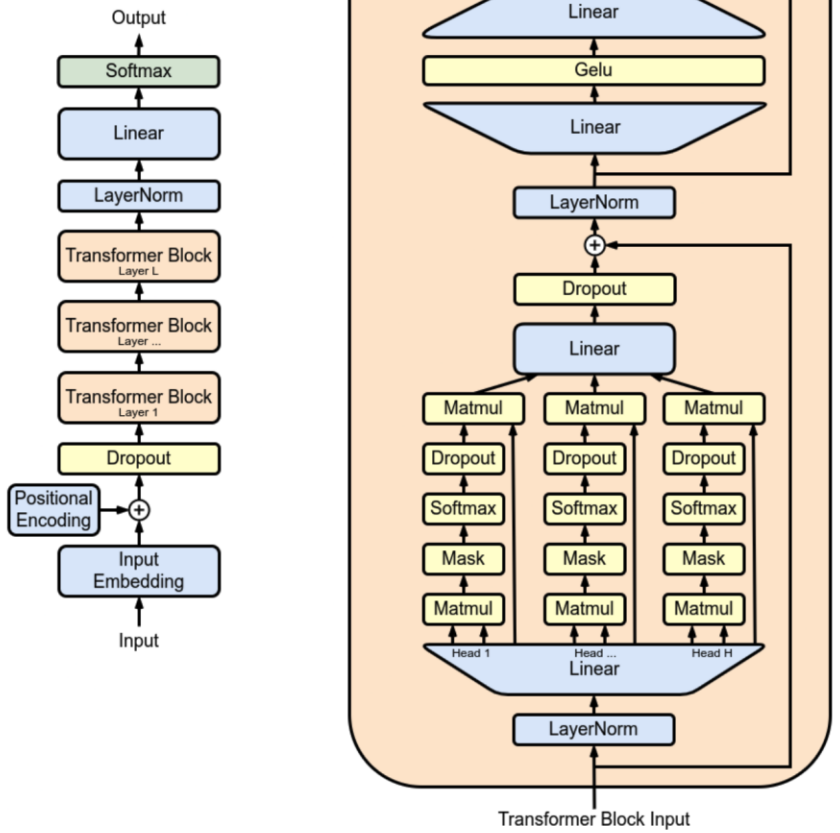
Fully Homomorphic Encryption (FHE) Can Solve LLM Privacy Challenges

The solution to the challenges of LLM deployment is to use ***Fully Homomorphic Encryption (FHE)*** which enables the execution of functions on encrypted data. It is possible to achieve the goal of protecting the ***model owner's IP*** while still maintaining the ***privacy of the user's data***. This demo shows that an LLM model implemented in FHE maintains the quality of the original model's predictions.

Model Encryption

Fully Homomorphic Encryption (FHE) Can Solve LLM Security Challenges

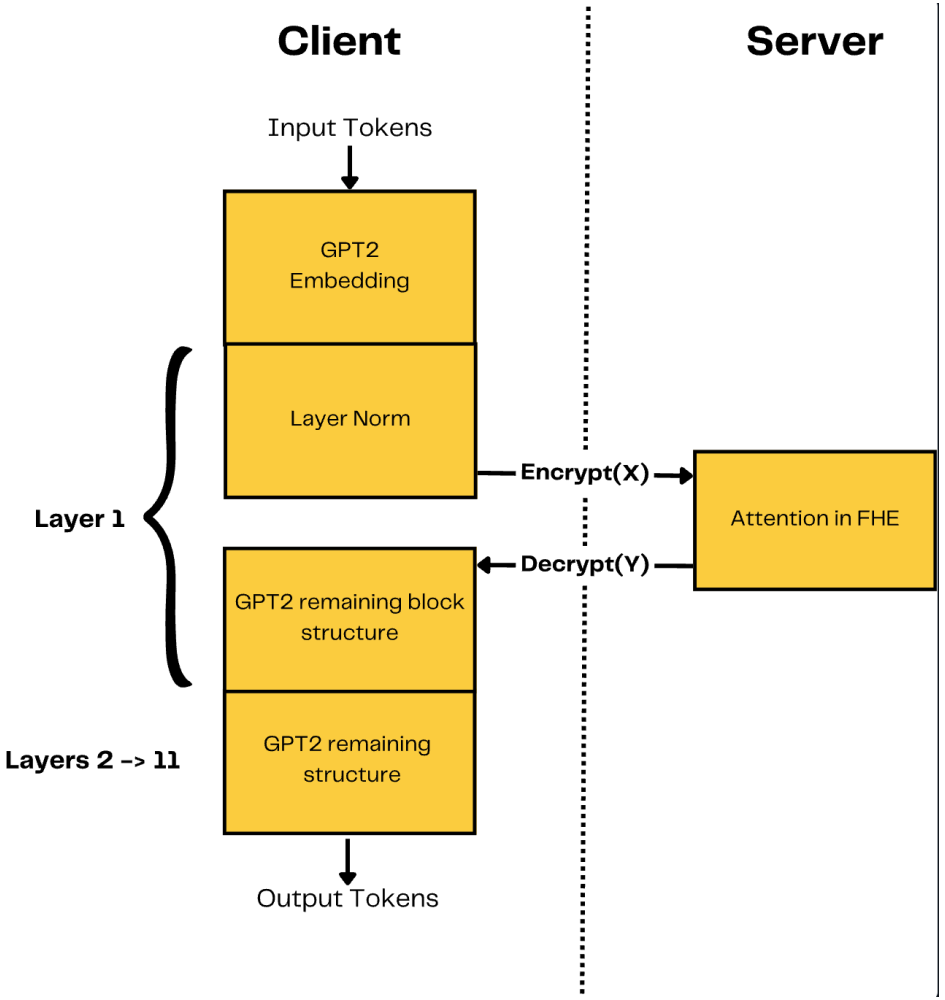
The model weights and activations are represented with integers. Nonlinear functions must be implemented with a **Programmable Bootstrapping (PBS)** operation. PBS implements a **table lookup (TLU)** operation on encrypted data while also refreshing ciphertexts to allow arbitrary computation



Model Encryption

Fully Homomorphic Encryption (FHE) Can Solve LLM Security Challenges

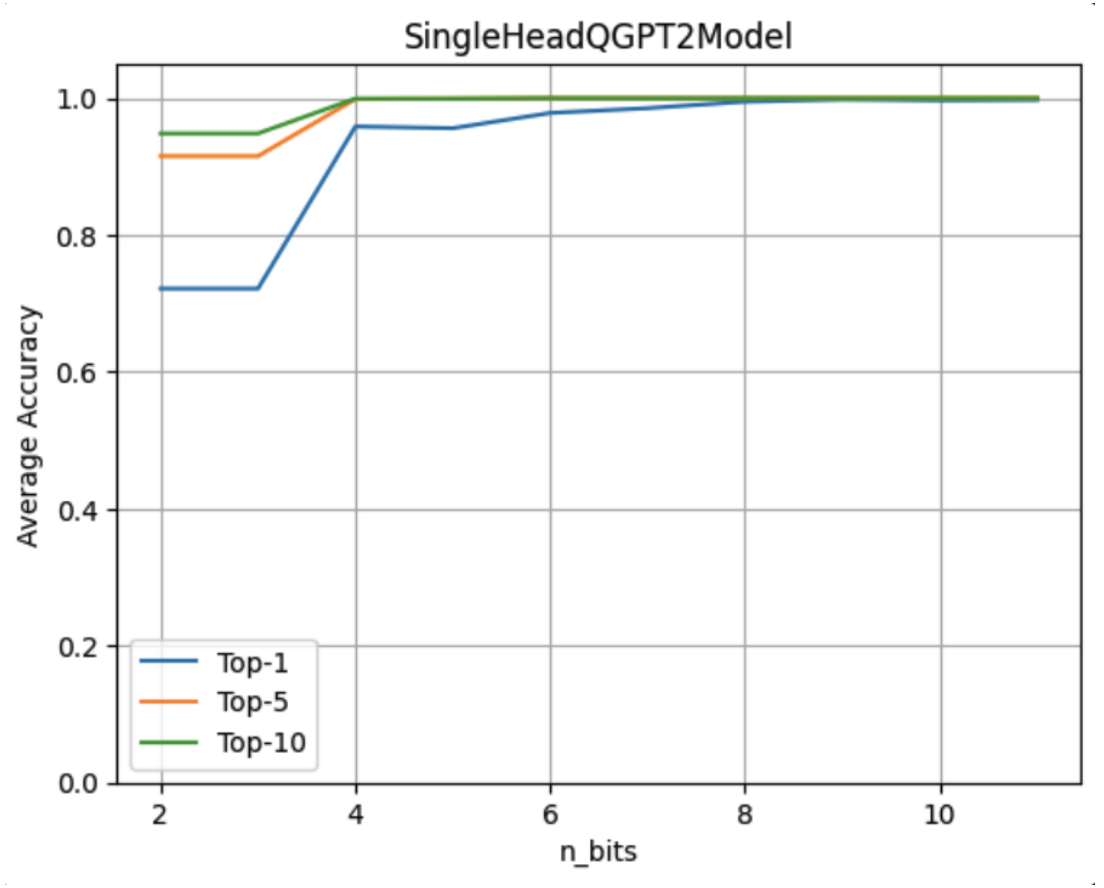
A simplified overview of the underlying implementation: **A client** starts the inference locally up to the ***first layer*** which has been removed from ***the shared model***. The user encrypts the intermediate operations and sends them to the server. The server applies ***part of the attention mechanism*** and the results are then returned to the client who can ***decrypt them and continue the local inference***.



Model Encryption

Fully Homomorphic Encryption (FHE) Can Solve LLM Security Challenges

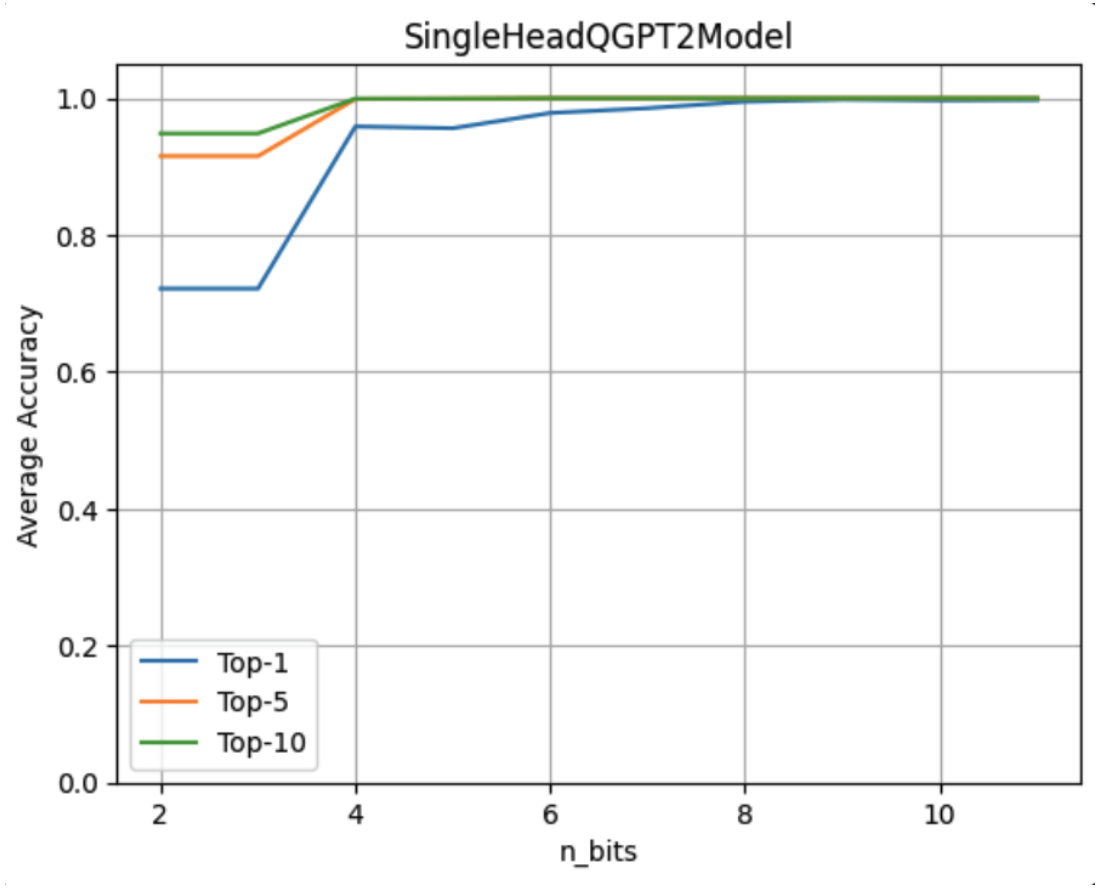
To evaluate the impact of quantization, run the ***full GPT2 model*** with ***a single LLM Head operating over encrypted data***. Then, evaluate the accuracy obtained when varying the number of quantization bits for both weights and activations.



Model Encryption

Fully Homomorphic Encryption (FHE) Can Solve LLM Security Challenges

The 4-bit quantization maintains 96% of the original accuracy. The experiment is done using a data-set of ~80 sentences. The metrics are computed by comparing the logits prediction from the original model against the model with the quantized head model.



Model Encryption

Fully Homomorphic Encryption (FHE) Can Solve LLM Security Challenges

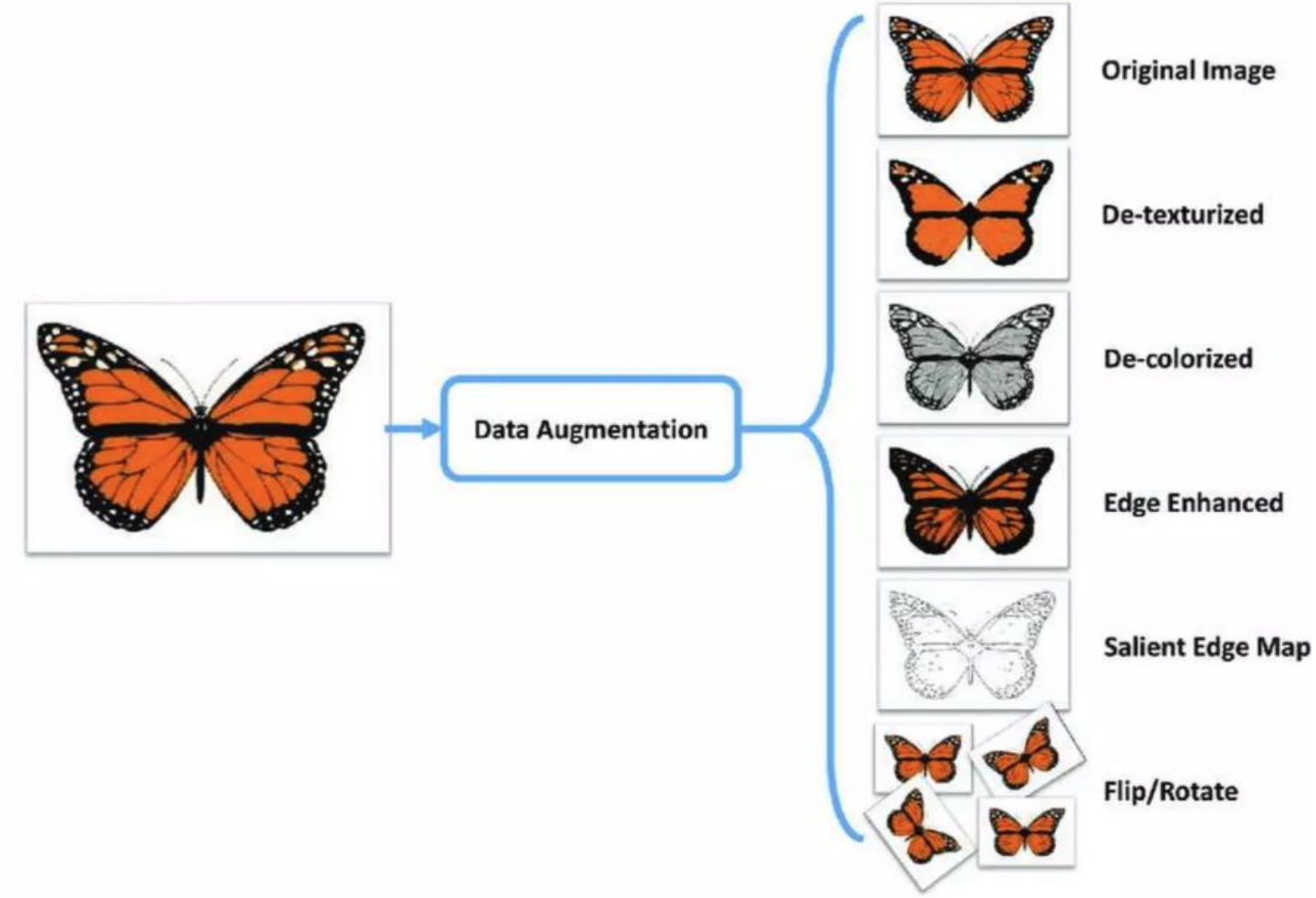
Demo code:

```python
class SingleHeadAttention(QGPT2):
    """Class representing a single attention head implemented with quantization methods."""


    def run_numpy(self, q_hidden_states: np.ndarray):

        # Convert the input to a DualArray instance
        q_x = DualArray(
            float_array=self.x_calib,
            int_array=q_hidden_states,
            quantizer=self.quantizer
        )

        # Extract the attention base module name
        mha_weights_name = f"transformer.h.{self.layer}.attn."

        # Extract the query, key and value weight and bias values using the proper indices
        head_0_indices = [
            list(range(i * self.n_embd, i * self.n_embd + self.head_dim))
            for i in range(3)
        ]
        q_qkv_weights = ...
        q_qkv_bias = ...
```

```python
        # Apply the first projection in order to extract Q, K and V as a single array
        q_qkv = q_x.linear(
            weight=q_qkv_weights,
            bias=q_qkv_bias,
            key=f"attention_qkv_proj_layer_{self.layer}",
        )

        # Extract the queries, keys and vales
        q_qkv = q_qkv.expand_dims(axis=1, key=f"unsqueeze_{self.layer}")
        q_q, q_k, q_v = q_qkv.enc_split(
            3,
            axis=-1,
            key=f"qkv_split_layer_{self.layer}"
        )

        # Compute attention mechanism
        q_y = self.attention(q_q, q_k, q_v)

        return self.finalize(q_y)
```

Robustness Enhancement

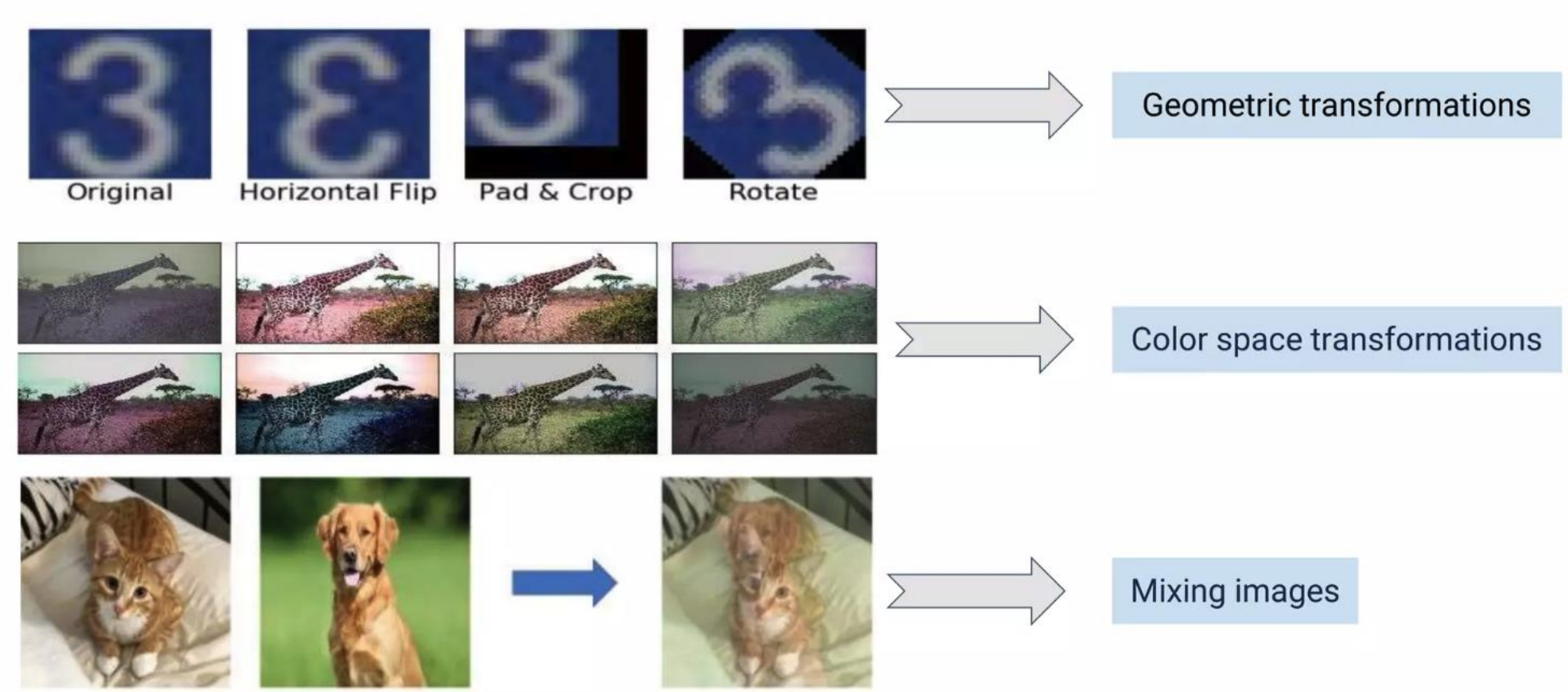
Data Augmentation



Robustness Enhancement

Data Augmentation

For Vision data: **Geometric transformations, Color space transformations, Mixing images**



Robustness Enhancement

Data Augmentation

For text data

In my presentation focus topic is Data Augmentation.

In my presentation **theme** topic is Data Augmentation -- Synonym Replacement

In my presentation focus topic is Data Augmentation **techniques** -- Random Insertion

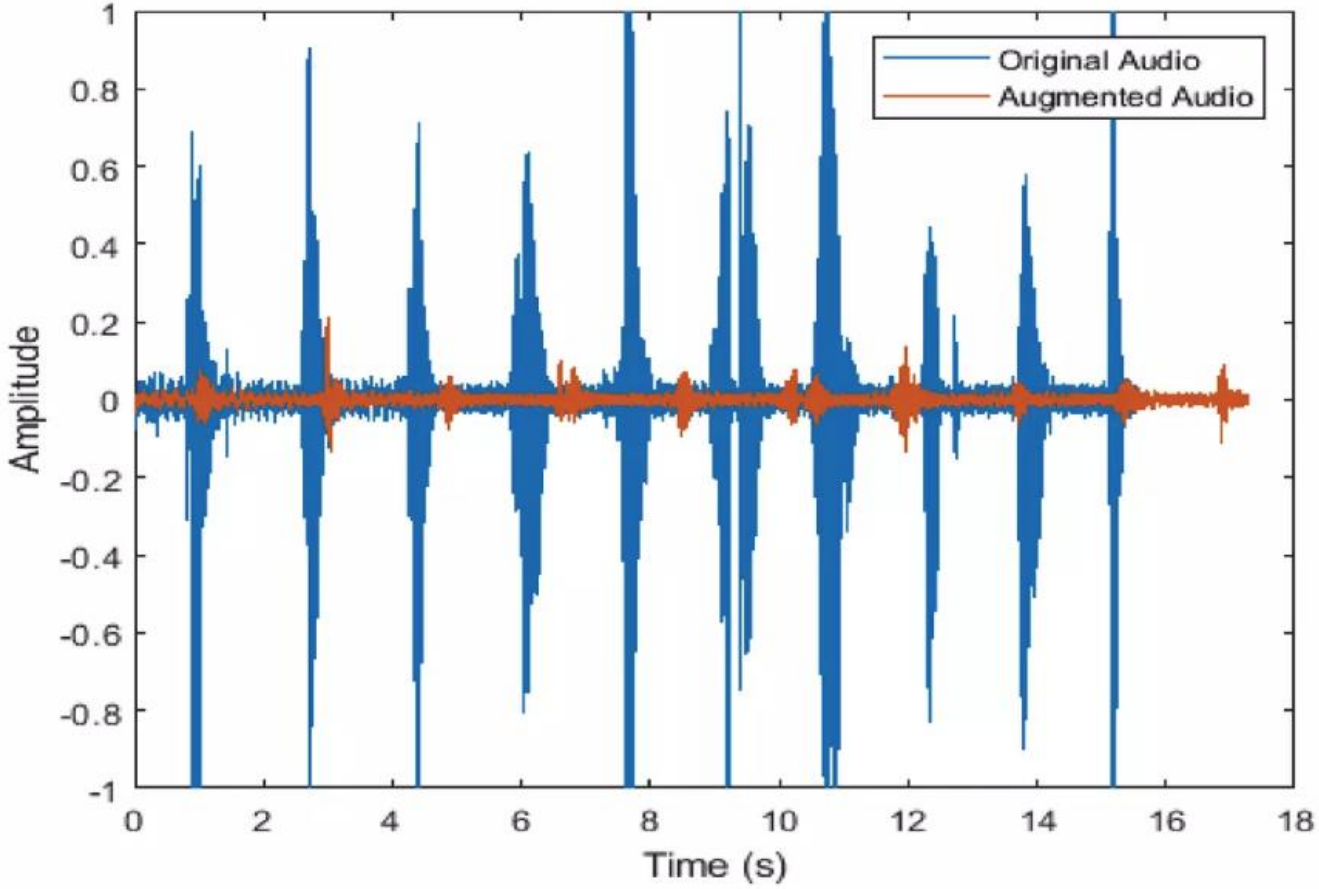
In my presentation **topic focus** is Data Augmentation -- Random Swap

In my presentation focus topic Data Augmentation --Random Deletion

Robustness Enhancement

Data Augmentation

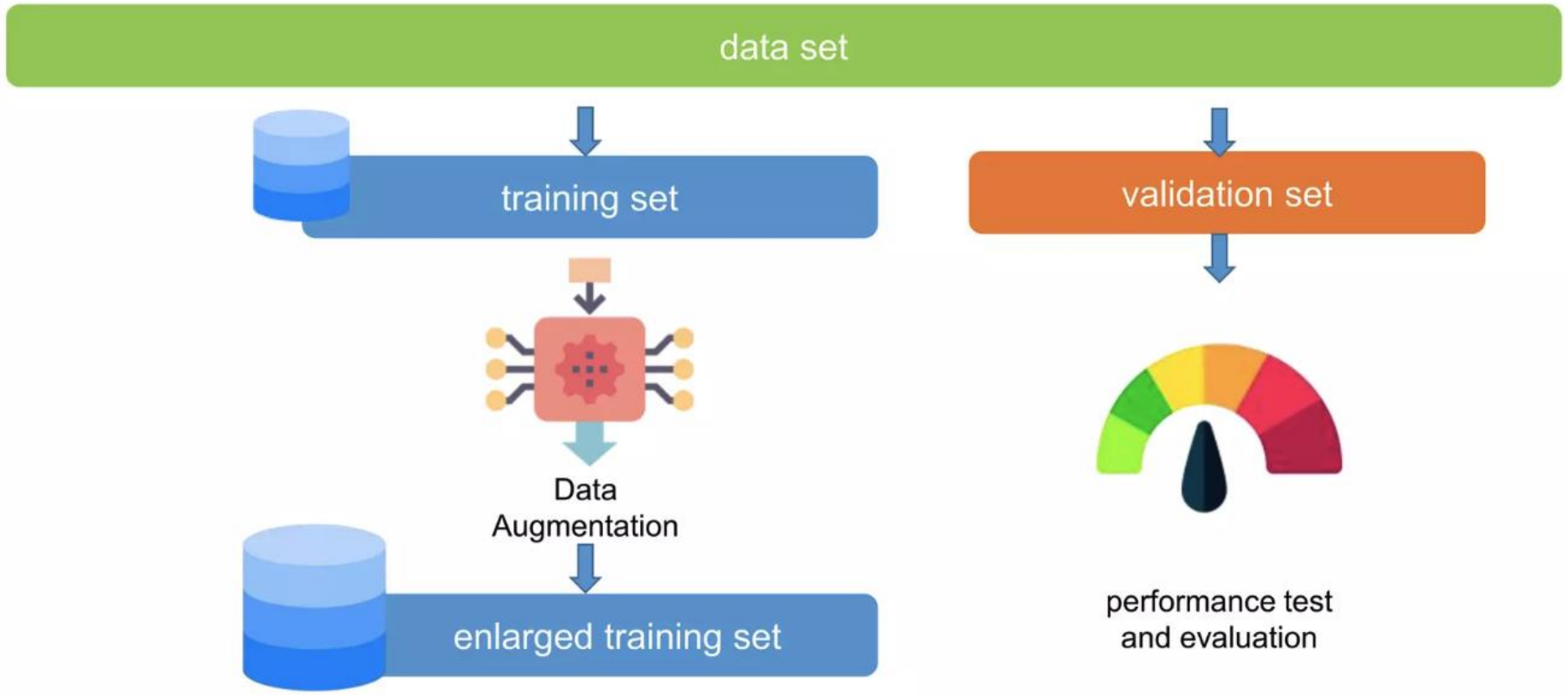
For Audio data: **Noise Injection, Shifting, Changing the speed of the tape.**



Robustness Enhancement

Data Augmentation

Workflow of Data Augmentation



Robustness Enhancement

Data Augmentation

Demo Code:

Albumentations is a Python library for fast and flexible image enhancement.Albumentations efficiently implements a rich variety of image transformation operations that are optimised for performance, while providing a clean and powerful image enhancement interface for different computer vision tasks, including object classification, segmentation and detection.

Robustness Enhancement

Data Augmentation

Geometric transformation

```
import cv2
import albumentations as A
from show_images import show_images

# 读取图像
image = cv2.imread("../images/example.jpg")
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# 定义几何变换
transform = A.Compose([
    A.HorizontalFlip(p=0.5),  # 水平翻转
    A.VerticalFlip(p=0.5),    # 垂直翻转
    A.Rotate(limit=90, p=0.5),  # 随机旋转
    A.RandomCrop(256, 256, p=0.5),  # 随机裁剪
    A.Resize(300, 300, p=1)  # 调整图像大小
])

# 应用变换
augmented = transform(image=image)
augmented_image = augmented['image']

show_images(image,augmented_image)
```

Robustness Enhancement

Data Augmentation

Color transformation

```python
import cv2
import albumentations as A

# 读取图像
image = cv2.imread("../images/example.jpg")
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# 定义颜色变换
transform = A.Compose([
    A.RandomBrightnessContrast(
        brightness_limit=0.2,
        contrast_limit=0.2,
        p=0.5
    ),  # 随机调整亮度和对比度
    A.HueSaturationValue(
        hue_shift_limit=20,
        sat_shift_limit=30,
        val_shift_limit=20,
        p=0.5
    ),  # 调整色调、饱和度和亮度
    A.RGBShift(
        r_shift_limit=20,
        g_shift_limit=20,
        b_shift_limit=20, p=0.5
        )  # 随机偏移 RGB 通道
])

# 应用变换
augmented = transform(image=image)
augmented_image = augmented['image']
```

Robustness Enhancement

Data Augmentation

Robustness Enhancement

Contrastive Learning

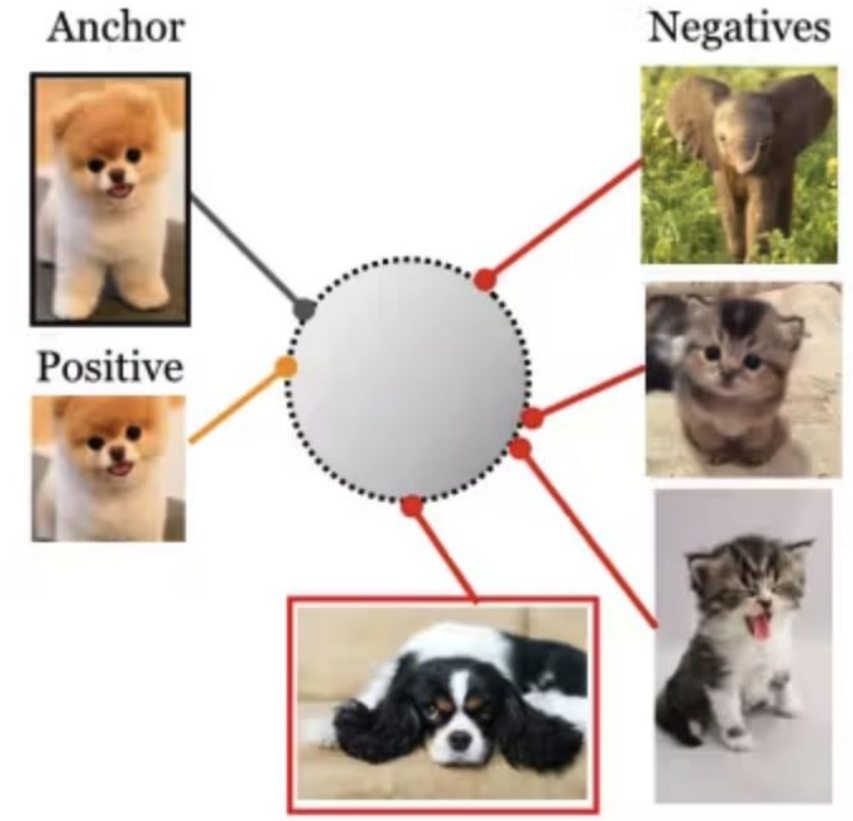
Contrastive learning, a powerful technique in the field of machine learning, significantly **enhances model robustness** by learning to distinguish between similar and dissimilar data points. This method involves training models on pairs or groups of examples, teaching them to recognize and amplify the subtle differences that define each class.

By focusing on these differences, models develop a deeper, more nuanced understanding of the data, leading to improved generalization across varied or unseen scenarios. Furthermore, ***contrastive learning encourages the model to form embeddings that are invariant to small perturbations or noise, thereby increasing its robustness against adversarial attacks and other forms of input variability.***

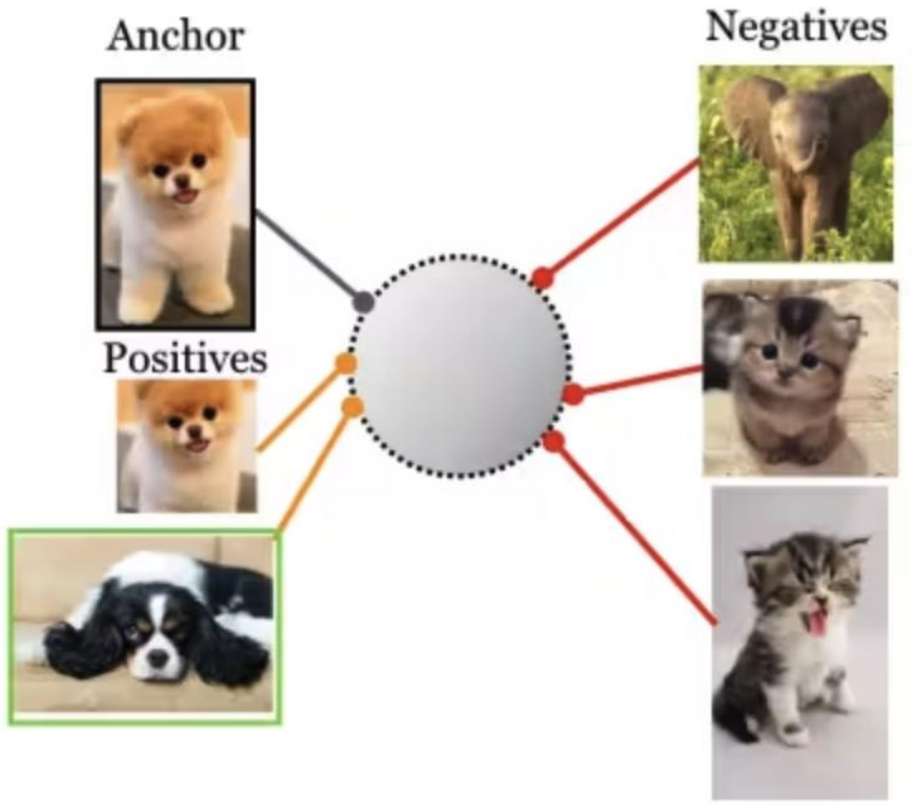
Robustness Enhancement

Contrastive Learning

Supervised Learning

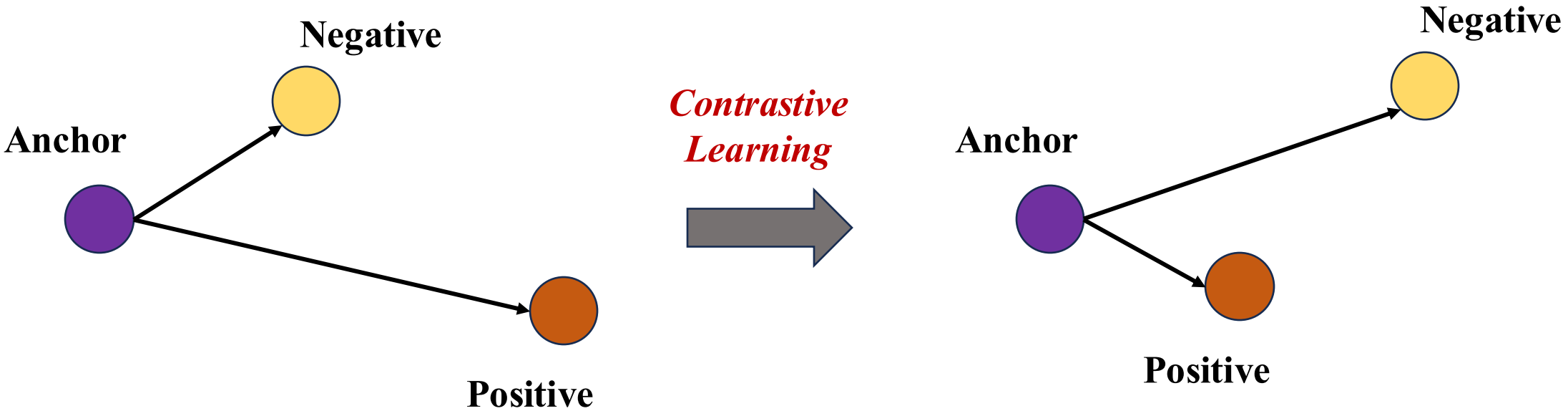


Contrastive Learning



Robustness Enhancement

Contrastive Learning

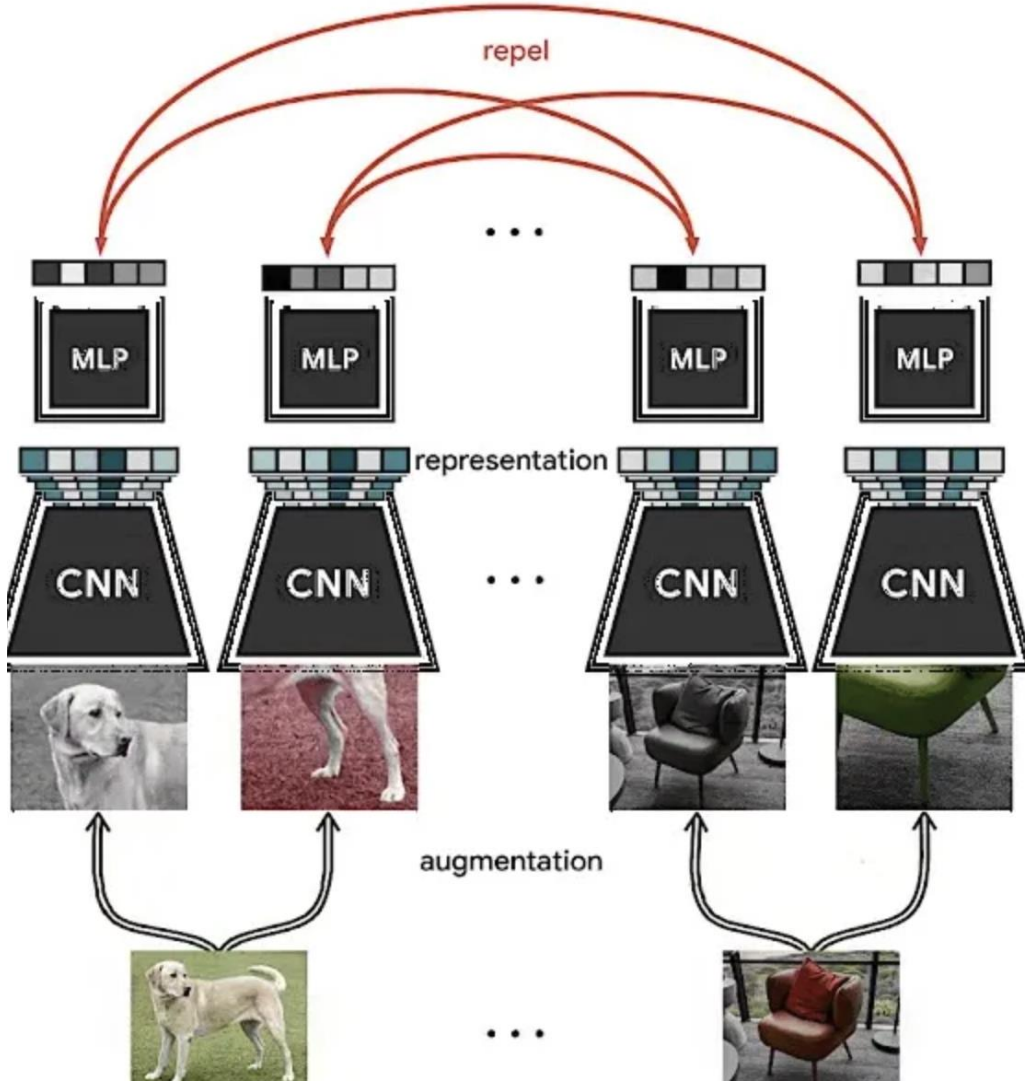


The key point is to create a 'ternary constraint' in which anchor instances are drawn closer to positive samples while being pushed away from negative samples. By learning a representation that satisfies the ternary constraint, the model can better distinguish between similar and dissimilar instances.

Robustness Enhancement

Contrastive Learning

SimCLR is a ***self-supervised contrast learning framework*** widely recognised for its effectiveness in learning powerful representations. It builds on the principles of contrast learning by utilising ***a combination of data augmentation, well-designed contrast objectives and symmetric neural network architectures***.



Quiz

Which of the following is a common technique used in data augmentation for images?

A) Deleting data

B) Rotating images

C) Reducing the size of the dataset

D) Compressing images

Quiz

What is the primary purpose of data augmentation?

A) To increase the computational cost of training

B) To artificially increase the size of the training dataset

C) To reduce the complexity of the model

D) To avoid using any data

Quiz

What is the main goal of contrastive learning in machine learning?

A) To maximize the similarity between different classes

B) To minimize the similarity between examples from the same class

C) To maximize the similarity between examples from the same class

D) To ignore the differences between examples

Quiz

In contrastive learning, what kind of pairs are typically compared to learn useful features?

A) Random pairs of examples

B) Only pairs of examples from different classes

C) Pairs of similar and dissimilar examples

D) Only pairs of examples from the same class

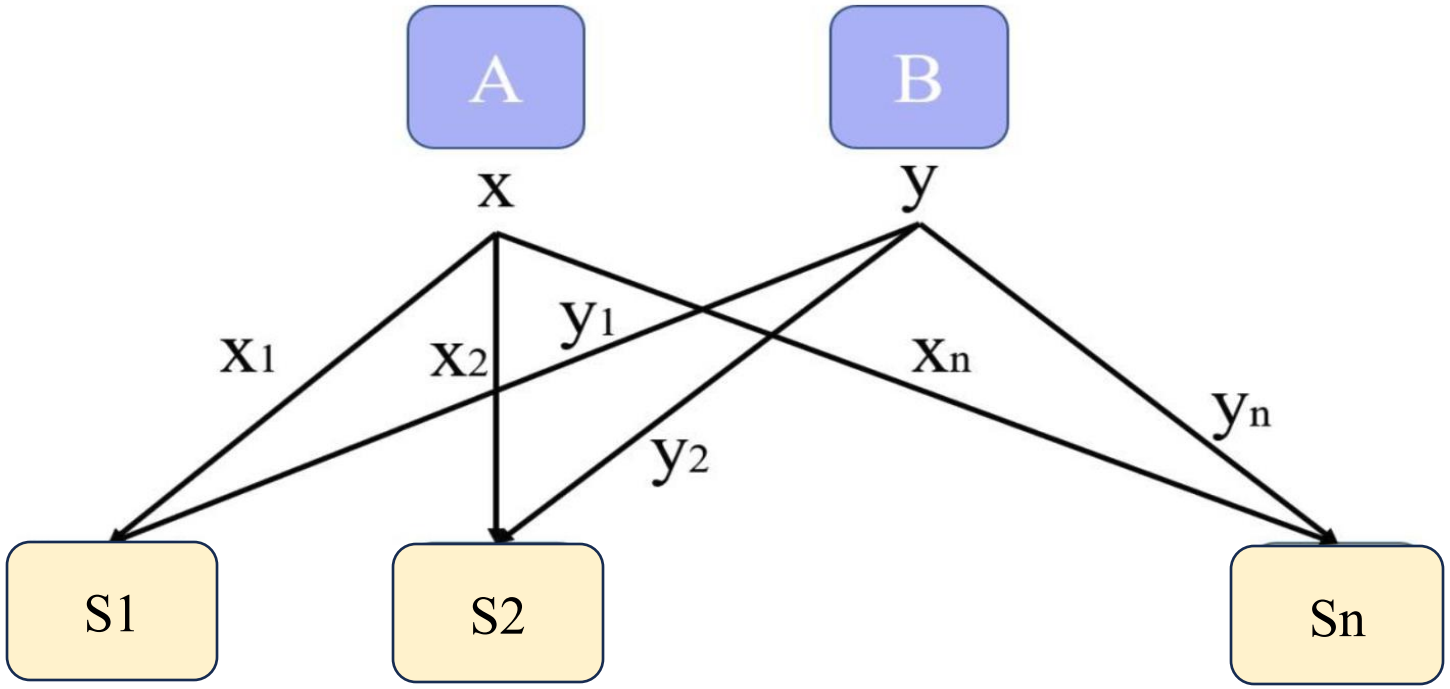
Secure Multi-Party Computation

The Secure Multi-party Computation problem (SMC, Secure Multi-party Computation) was proposed by **Professor Yao Qizhi**, a Chinese computer scientist and the winner of the 2000 Turing Award, in his paper 'Protocols for secure computations' in 1982 as the Millionaire's Problem (two millionaires Alice (***two millionaires Alice and Bob want to know who is richer of the two of them, but neither of them wants the other and other third parties to know any information about their wealth***) proposed, opened up a new field of cryptography research.

Secure Multi-Party Computation

Secure Multi-Party Computation

Definition of Secure Multi-Party Computation: Participants A and B have secret values x and y, respectively, and need to compute the $z = f(x, y)$, but do not want to reveal the secret value to each other.A and B use the secret partitioning algorithm on the secret value to obtain the shares $(x_1, x_2 \cdots x_n)$ and $(y_1, y_2 \cdots y_n)$ respectively. Send the share to the node $s_1, s_2 \cdots s_n$. The node computes s_i . Get enough z_i to execute the secret reorganisation algorithm to get z.



Secure Multi-Party Computation

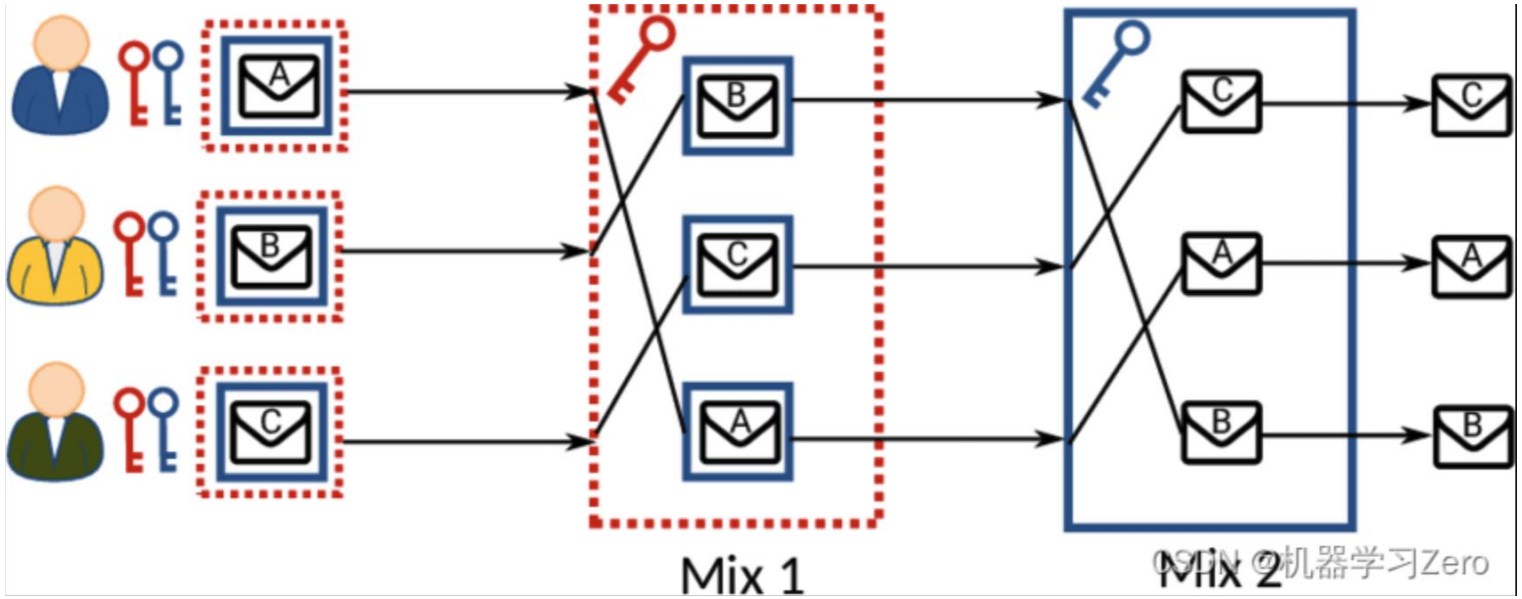
Bit Commitment Scheme:

An intuitive example of a bit promise: Alice puts the message M (the promise) in a box (only Alice has the key) and gives it to Bob under lock and key; when Alice decides to confirm the message M to Bob, Alice gives Bob the message M and the key; Bob is able to open the box and verify that the message in the box is the same as the one presented by Alice, and Bob is sure that the message in the box is the same as the one presented by Alice. Bob is able to open the box and verify that the message in the box is the same as the message presented by Alice, and Bob is confident that the message in the box has not been tampered with while in his possession.

Secure Multi-Party Computation

Mix-Network:

A mix-network is a multi-party protocol whose input is ***a table of ciphertexts*** that have a ***one-to-one*** correspondence with plaintexts. The mix-network ***randomly*** replaces this ciphertext table and outputs another ciphertext table, called Blinded table. Like the input ciphertext table, the ciphertexts in the output blind table and the same set of plaintexts are also ***one-to-one correspondence***, i.e., the output ciphertext table is a random replacement of the input ciphertext table. The security of the mix-network lies in the fact that an attacker cannot determine which ciphertext in the output blind table corresponds to which ciphertext in the input ciphertext table.



Secure Multi-Party Computation

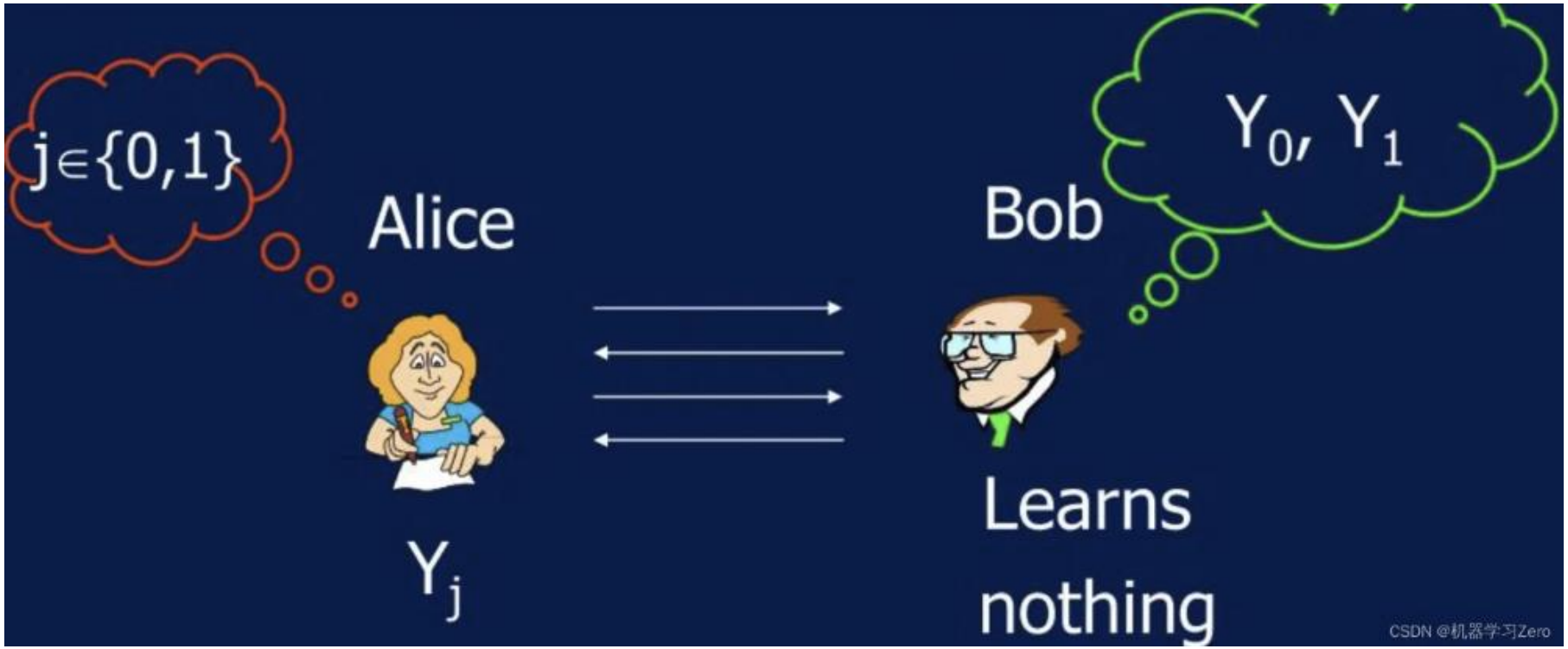
Oblivious Transfer:

Oblivious Transfer is a method of secretly obtaining a ***portion of a message*** from a collection of messages, and after the protocol is executed, the receiver knows ***whether or not*** he received the secret, but the sender does not (oblivious).

Secure Multi-Party Computation

Oblivious Transfer:

Consider the scenario where A intends to sell the answers to many questions, and B intends to buy the answer to one of the questions, but does not want A to know which question he bought. I.e., B does not want to give away to A the secret of which question he is in possession of, and such a scenario can be realised by an oblivious transmission protocol.



Adversarial Attack Experiment:

- Generate adversarial samples using the **FGSM method**. This involves slightly modifying the original images so that the model misclassifies these modified images.
- Students need to observe and record the difference in the model's response to normal and adversarial images.

Practice: Adversarial Attack Experiment

```python
import torch
import torchvision.models as models
import torchvision.transforms as transforms
from torchvision.datasets import CIFAR10
from torch.utils.data import DataLoader
import torch.nn.functional as F

# Loading pre-trained ResNet models
model = models.resnet18(pretrained=True)
model.eval()  # Setting the model to evaluation mode
```

Practice: Adversarial Attack Experiment


```
12    # load data
13    transform = transforms.Compose([
14        transforms.ToTensor(),
15    ])
16    dataset = CIFAR10(root='./data', train=False, download=True, transform=transform)
17    loader = DataLoader(dataset, batch_size=1, shuffle=True)
18
19    # get one image and label
20    dataiter = iter(loader)
21    images, labels = next(dataiter)
```

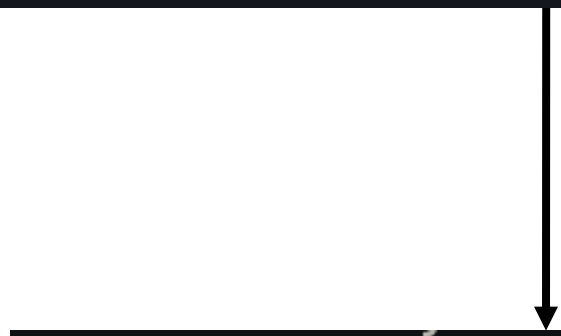
Practice: Adversarial Attack Experiment


```python
# adversarial attack: FGSM
def fgsm_attack(image, epsilon, data_grad):
    # collect the element symbol of data gradient
    sign_data_grad = data_grad.sign()
    # create perturbed image by adjusting each pixel of the input image
    perturbed_image = image + epsilon * sign_data_grad
    # add clipping to maintain [0,1] range
    perturbed_image = torch.clamp(perturbed_image, 0, 1)
    return perturbed_image
```

Practice: Adversarial Attack Experiment

```python
# loss function and optimizer
criterion = torch.nn.CrossEntropyLoss()

# zero all the gradients
images.requires_grad = True

# forward propagation
outputs = model(images)
loss = criterion(outputs, labels)
model.zero_grad()

# backward propagation
loss.backward()

# collect data gradient (for FGSM)
data_grad = images.grad.data

# call FGSM Attack
epsilon = 0.1  # can adjust epsilon to see different effects
perturbed_data = fgsm_attack(images, epsilon, data_grad)

# reclassify perturbed image
output = model(perturbed_data)
```

```python
# check success rate
final_pred = output.max(1, keepdim=True)[1]  # get the index of the maximum log-probability
correct = final_pred.eq(labels.view_as(final_pred)).sum().item()

print('original label:', labels)
print('predicted label:', final_pred)
print('attack success:', labels != final_pred)
```



```
original label: tensor([1])
predicted label: tensor([[922]])
attack success: tensor([[True]])
```

THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

AIAA 2290: Ethics, Privacy and Security in AI

Thanks!!

Xuming HU

xuminghu@hkust-gz.edu.cn

The Hong Kong University of Science and Technology (Guangzhou)

2025 Fall